

Distributed MapReduce on Lab Machines

SLR Project P4

Project presentation

10-minute discussion

One-sentence summary

A custom Go MapReduce framework running on *.enst.fr workers — direct Common Crawl input, pluggable jobs, local-disk shuffle, fault-aware coordination, no root, no Docker.

What We Built

- Host discovery from `tp.telecom-paris.fr/ajax.php` — live JSON, canonical `.enst.fr` names, $2 \times N$ pool for cold spares
- Go orchestrator: build worker binary, deploy, coordinate phases, merge results
- HTTP worker API: `/load` $\rightarrow$ `/map` $\rightarrow$ `/intermediate` $\rightarrow$ `/reduce` $\rightarrow$ `/result`
- Four pluggable jobs sharing the same framework: `wordcount`, `langdetect`, `domainpop`, `doccdensity`
- 89 Go tests; reference wordcount matched exactly against sequential implementation

Key Design Choices

Storage: everything avoids NFS

- Input: streamed directly from `data.commoncrawl.org` (Amazon S3) — `/cal/commoncrawl` never read
- Intermediate bucket files: `os.MkdirTemp` → `/tmp` on each worker
- Worker binary: SCPed to `/tmp/mr-worker`, not `$HOME`
- Final output: written locally on the orchestrator

Deployment: N SCP to `/tmp`

- Binary cross-compiled locally (`GOOS=linux` `GOARCH=amd64`)
- N SCPs directly to `/tmp` (vs. rubric's 1 SCP to NFS home)
- Trade-off: avoids NFS entirely; acceptable for $N \leq 20$
- SSH multiplexing (ControlMaster): SCP and start share one TCP connection
- `BatchMode=yes`,
`StrictHostKeyChecking=no`,
`ConnectTimeout=10`

- 1 **Load** — Common Crawl URLs via POST /load; workers download WET files to /tmp
- 2 **Map** — each worker writes N bucket files on local disk (/tmp)
- 3 **Shuffle+Reduce** — reducer i pulls bucket i from every peer via GET /intermediate (P2P, no orchestrator)
- 4 **Collect** — orchestrator fetches /result, merges, sorts, writes output

Partitioning

$\text{FNV-32a}(\text{key}) \% N$

All values for the same key always route to the same reducer. N stays **fixed** for the job — changing it mid-run would split the value list across two reducers.

Bootstrap

Workers start independently. Peers are discovered only when the orchestrator sends POST /reduce {peers:[...]}.

Detection

- Health watcher polls `GET /health` every 5 s
- Host declared dead after **2 consecutive failures**

Recovery

- Slot $\rightarrow$ spare host; slot state reset to pending
- Prerequisites replayed: `load` $\rightarrow$ `map` $\rightarrow$ `reduce`
- **Epoch** incremented: any reduce at old epoch is rewound and re-run (peer set changed)

Atomic writes

WET downloads: `CreateTemp` + `os.Rename` (ok)

Binary deploy: `mv` on same filesystem (ok)

Final output: direct write (known gap)

Known gaps

No speculative execution for stragglers.

Orchestrator crash $\Rightarrow$ job must restart (workers survive via `nohup`).

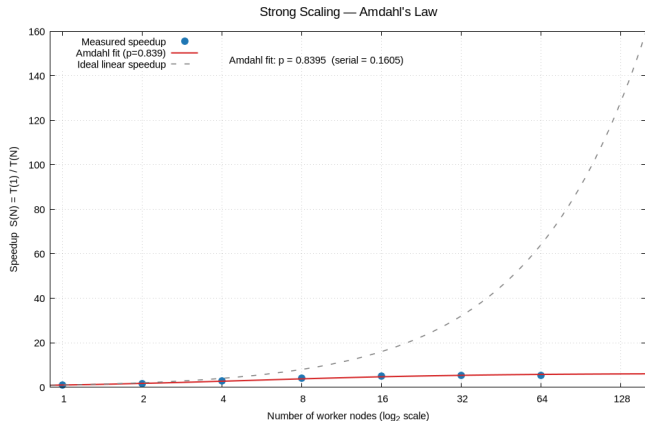
Use Cases Beyond Wordcount

- **langdetect** — stop-word scoring per line (EN/FR/DE/ES/IT); WARC headers skipped; result: language share across the crawl. *Common Crawl is $\approx 60\%$ English.*
- **domainpop** — reads WARC-Target-URI: header per WET record; counts pages per domain; result: global domain popularity ranking (*youtube.com*, *wikipedia.org*, *doi.org* top-ranked). Quality signal: unexpectedly high-ranked spam domains flag filtering needs.
- **docrdensity** — counts chars, alphanumeric ratio, and line-length histogram per content line; result: $\approx 77\%$ alphanumeric, most lines < 80 chars, very long lines = minified JS/CSS.

Why Common Crawl matters

One of the primary training data sources for GPT, LLaMA, Mistral and most modern LLMs. These jobs are the kind of analysis that precedes any large-scale dataset curation.

Measurements and Amdahl's Law



- Same 8 WET files at every point (correct methodology)
- 1 node: 206.95 s $\Rightarrow$ baseline
- 16 nodes: 40.80 s, speedup 5.07
- 64 nodes: 39.06 s — *regression*

Serial fraction

Plateau $S \approx 5.45 \Rightarrow f \approx 18\%$ serial.
Source: orchestrator collect phase (sequential merge of N results) and $N \times (N-1)$ HTTP connections at 64 nodes.

Three-Way Comparison

	Our MapReduce	Hadoop	Kafka Streams
Model	Batch, 1 stage	Batch, multi-stage	Stream, unbounded
Storage	/tmp, no replication	HDFS 3× replication	Replicated broker log
Data locality	No — HTTP pull	Yes — run where data lives	Broker partitions
Stragglers	Not handled	Speculative execution	N/A
8-node wordcount	0.27 s	not run	10.1 s

Biggest gaps vs. Hadoop

(1) HDFS data locality — we transfer all input at load time. (2) Speculative execution — one slow node blocks the whole phase.

What's Missing + Demo

Real remaining gaps

- Atomic final output write (temp+rename)
- Speculative execution for stragglers
- Port collision preflight check
- Multi-chunk scheduling in local-file mode
- Cross-login validation

Live demo

```
cd scripts && go test ./...  
./mapreduce.sh \  
-job scripts/jobs/wordcount \  
-input test_input.txt \  
-n 4 -output result.txt  
./run_all_commoncrawl_jobs.sh \  
-crawl CC-MAIN-2026-21 \  
-files-limit 1 -n 4
```

Backup if machines are unstable

Passed tests, checked-in wordcount output (24M unique words), Amdahl plot, timing CSVs